World of Promptcraft

LangGraph-Powered NPCs in a 3D RPG

Type anything. The dragon reacts.

LLMDays Hamburg - 2025

LangGraph | FastAPI | Three.js | TypeScript | Python

github.com/Zaexv/world-of-promptcraft

The Idea

No buttons. No scripted trees. Just text.

- You open a 3D world rendered in your browser (Three.js + WebGL).
- Walk up to an NPC - a dragon, a merchant, a wandering knight.
- Type anything in the chat box. Free-form. Unscripted.
- The NPC reasons, acts, and responds in real-time.

Classic approach

[x] Dialogue trees
[x] Scripted if/else
[x] Hard-coded responses
[x] Stale after 1 hour

Agentic approach

[ok] LLM reasons each turn
[ok] Tool calls = real game effects
[ok] Persistent memory + mood
[ok] Every conversation unique

Architecture Overview

Full Stack

Client
Three.js + TypeScript + Vite

Transport
WebSocket (port 8000)

Server
FastAPI + Python 3.11+

Agents
LangGraph StateGraph per NPC

LLM
Claude / OpenAI / Ollama

Key Principles

Server-authoritative
WorldState lives on server; client is a render mirror

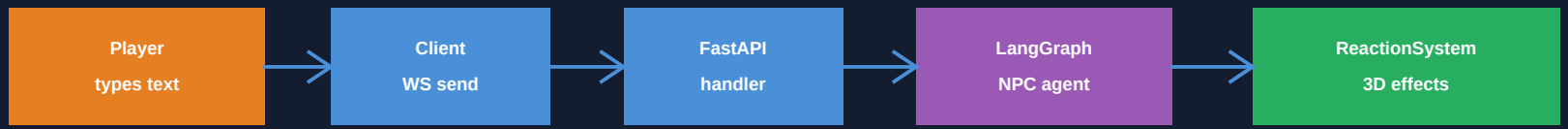
One graph per NPC
Each NPC = independent LangGraph StateGraph

Tool-driven actions
LLM calls typed functions -> real game effects

Persistent memory
MemorySaver per (npc_id, player_id) thread

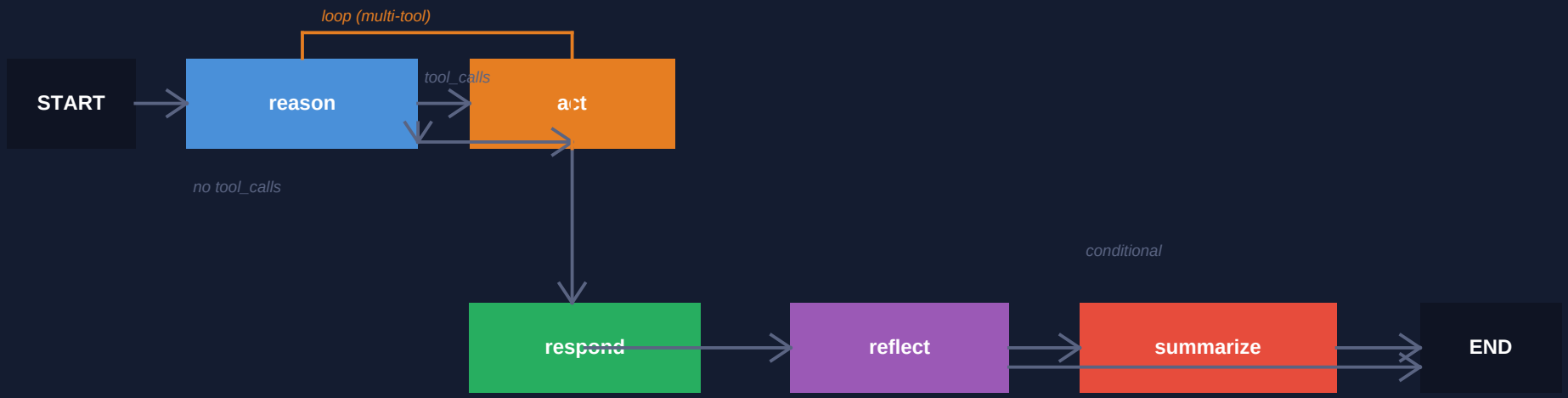
Prompt as interface
No buttons - text IS the game mechanic

WebSocket Message Flow



actions[] flow back: damage, emotes, weather, quests, items

The LangGraph Pipeline



reason

LLM CALL

Builds system prompt, injects world context + memory + RAG lore.
Binds 14 tools. If LLM emits tool_calls -> route to act.

act

NO LLM

Executes tool calls. Appends action dicts to pending_actions[].
Loops back to reason for multi-step reasoning.

respond

NO LLM

Extracts dialogue text from last AIMessage. Passes pending_actions through.

reflect

NO LLM

Heuristic keyword analysis. Updates mood, relationship_score,
and personality_notes. Zero cost.

summarize

CONDITIONAL

LLM call only every 3rd turn after 10 exchanges. Compresses memory
to 2-3 sentences. The only 2nd LLM call in the pipeline.

Live Demo Trace - Ignathar the Dragon

Player types: "I challenge you, wyrm!"

1 reason

```
System prompt includes: "On EVERY combat, MUST call deal_damage (20-50 fire)
  AND spawn_effect(fire). Below 250 HP: deal 40-50 instead."
LLM sees: Ignathar HP = 180  ->  below 250 threshold  ->  enrage mode
->  AIMessage with tool_calls: [deal_damage(45, fire), spawn_effect(fire), change_weather(storm)]
```

2 act

```
deal_damage  -> pending_actions += {kind:damage, amount:45, type:fire}
             -> world_state[player][hp] = 100 - 45 = 55
spawn_effect -> pending_actions += {kind:spawn_effect, effectType:fire}
change_weather -> pending_actions += {kind:change_weather, weather:storm}
```

3 reason (round 2)

```
LLM sees ToolMessages confirming all 3 tools succeeded
->  AIMessage (no tool_calls): "THOU DAREST CHALLENGE ME?! FEEL THE FURY..."
```

4 respond

```
response_text = "THOU DAREST CHALLENGE ME?! FEEL THE FURY OF A THOUSAND EMBERS!"
```

5 reflect

```
tokens {challenge, wyrm} -> hostile_count=1 -> mood stays "angry"
relationship delta: damage(-5) + hostile word(-2) = -7
relationship_score = -12  ->  tier: DISTRUSTFUL
```


Tool System - LLM Calls Typed Functions

Each NPC's tools are closures over that NPC's pending_actions list and world snapshot. The LLM never touches game state directly - it calls a typed function that does it.

Combat	Dialogue	Trade	Environment	World Query	Quest
deal_damage	emote	offer_item	change_weather	get_nearby_entities	start_quest
defend	give_quest	take_item	spawn_effect	check_player_state	advance_objective
flee	complete_quest		move_npc		check_player_quests
heal_target					

The Closure Pattern

```
def get_all_tools(pending_actions: list, world_snapshot: dict) -> list[BaseTool]:
    @tool
    def deal_damage(target: str, amount: int, damage_type: str) -> str:
        """Deal damage to a target."""
        pending_actions.append({"kind": "damage", "params": {...}})
        world_snapshot["player"]["hp"] -= amount          # immediate snapshot update
        return f"Dealt {amount} {damage_type} damage to {target}"

    return [deal_damage, heal_target, emote, offer_item, ...]  # 14 tools total
```

Memory & Relationship System

How NPCs Remember You

- thread_id = "{npc_id}_{player_id}"
- Two players -> same NPC -> independent memories
- MemorySaver checkpoints after every invocation
- Persistent fields survive indefinitely across sessions

Persistent fields in NPCAgentState

`conversation_summary`	Rolling 2-3 sentence LLM summary of past conversations
`mood`	neutral / happy / angry / sad / fearful
`relationship_score`	-100 (enemy) to +100 (trusted ally)
`personality_notes`	NPC observations about this player - max 300 chars

Relationship Tiers

<= -50	ENEMY	Hostile, refuses interaction
-50->-10	DISTRUSTFUL	Wary, curt replies
-10->+10	STRANGER	Polite, reserved
+10->+50	FRIEND	Warm, helpful
> +50	TRUSTED ALLY	Shares secrets, rare quests

reflect node - zero-cost heuristic (no LLM call)

```
# keyword sets drive mood + relationship delta
hostile_tokens = {"attack", "kill", "challenge", "fight", "die", "curse", ...}
friendly_tokens = {"help", "please", "thank", "friend", "praise", ...}

delta = 0
if any(t in text for t in hostile_tokens):   delta -= 2
if any(t in text for t in friendly_tokens):  delta += 3
```

Cost Efficiency

Cheaper than it looks - by design

1 LLM call per turn

The reason node is the only mandatory call.
Every other node is free.

Conditional summarize

Only fires at human_count >= 10 AND every 3rd turn after that.

Heuristic reflect

Mood + relationship update via keyword token sets. Zero tokens.

RAG lore keyword

Top-3 keyword-matched lore snippets.
No embedding model needed.

Context bounded

Reason node keeps only a small recent
window + compact summary.

Typical token budget per player interaction

System prompt (personality + world ctx)	**~400-600 tokens**
Conversation window (last 6-8 turns)	**~300-500 tokens**
RAG lore injection (top-3 snippets)	**~100-200 tokens**
Tool schemas (14 tools)	**~800-1000 tokens**
Total input estimate	**~1600-2300 tokens**
Output (dialogue + tool_calls)	**~100-300 tokens**

Summarize node compresses after every 3rd turn post-10 exchanges, keeping the conversation_summary <= 500 chars. This keeps per-turn cost essentially flat over arbitrarily long player relationships.

The NPCs - Same Graph, Different Personalities

Every NPC shares the SAME LangGraph topology. Differentiation comes entirely from the system_prompt in the reason node. Three layers stack inside every prompt:

- Personality / voice (who am I, how do I speak?)
- _TOOL_RULES_PREAMBLE (shared: MUST use tools, never text alone)
- NPC-specific tool rules (Ignathar: deal 40-50 fire when enraged; Aurelia: offer_item on greeting)

Ignathar
Hostile Boss
HP 500 - dragon_01

Aurelia
Merchant
HP 100 - merchant_01

Nireg Jenkins
Oracle / God
HP 5000 - nireg_jenkins

Captain Rolan
City Guard
HP 180 - guard_malaka_01

Sister Constanza
Healer
HP 120 - healer_malaka_01

Paco el Churrero
Street Merchant
HP 100 - churrero_01

Sylvara
Wraith Monster
HP 70 - wraith_01

Tutorial-Man
Guide NPC
HP 1000 - tutorial_01

Agentic Design Patterns

One agent per entity

Each NPC = independent StateGraph + MemorySaver. No shared graph state leaks between NPCs. Scales horizontally. Bounded by a global semaphore for LLM concurrency.

Closure-scoped tool state

Tools close over their NPC's pending_actions list at startup. The LLM sees clean function signatures. Side effects are invisible to the LLM - it only reads return strings.

Deferred side effects

pending_actions is applied to authoritative WorldState ONLY after the full graph finishes. This ensures consistent server state even when act -> reason loops multiple times.

Layered memory without a database

MemorySaver = in-process checkpointing. conversation_summary keeps context bounded. reflect runs heuristics so mood/relationship update every turn at zero LLM cost.

Prompt-as-config for behaviour

NPC tool rules live in the system prompt, not in graph structure. Adding a new behaviour pattern = edit one string. No code changes, no graph rewiring needed.

Hybrid retrieval (keyword RAG)

LoreRetriever uses keyword matching (no embedding model). Fast, deterministic, cheap. Good enough for lore injection; upgrading to dense retrieval is a drop-in swap.

Key Takeaways

What this project shows about building with LLMs

- **Text IS the interface. No buttons needed when the LLM is the game engine.**
- **One graph per agent, not one graph for all. Isolation > complexity.**
- **Tool calls = structured side effects. LLM decides what, code decides how.**
- **Memory without a database. In-process MemorySaver + heuristic reflect node.**
- **Cost is predictable. 1 LLM call per turn + conditional summarize every ~3rd.**
- **Behaviour via prompt, not code. NPC tool rules live in the system prompt.**

github.com/Zaexv/world-of-prompcraft

Stack: LangGraph - FastAPI - Three.js - TypeScript - Python | LLMDays Hamburg 2025